llm-benchmarking-py

A comprehensive collection of Python functions designed to benchmark LLM (Large Language Model) projects and code generation capabilities. This library provides a diverse set of computational tasks across multiple domains to evaluate performance, correctness, and efficiency.

Overview

This benchmarking suite tests various aspects of code generation and execution including:
- Algorithm implementation (prime numbers, sorting)
- Control flow structures (loops, conditionals)
- Data structure operations (lists, arrays)
- String manipulation
- SQL query execution
- Data generation utilities

Features

🔢 Algorithms (`llm_benchmark.algorithms`)
- **Primes**: Prime number detection, prime summation, and prime factorization
- **Sort**: Sorting algorithms, Dutch flag partition, and finding max N elements

🔄 Control Flow (`llm_benchmark.control`)
- **SingleForLoop**: Single-loop operations for range sums, list maximums, and modulus operations
- **DoubleForLoop**: Nested loop operations for matrix sums, pair counting, and duplicate detection

📊 Data Structures (`llm_benchmark.datastructures`)
- **DsList**: List manipulation including modify, search, sort, reverse, rotate, and merge operations

🔤 String Operations (`llm_benchmark.strings`)
- **StrOps**: String reversal and palindrome detection

🗄️ SQL Queries (`llm_benchmark.sql`)
- **SqlQuery**: Database operations including album queries, table joins, and invoice analysis using SQLite (Chinook database)

🎲 Generators (`llm_benchmark.generator`)
- **GenList**: Random list and matrix generation for testing purposes

Installation

Prerequisites
- Python 3.8 or higher
- Poetry (Python package manager)

Setup

1. Clone the repository:
```bash
git clone <repository-url>
cd llm-benchmarking-py
```

2. Install dependencies:
```bash
poetry install
```

Usage

Running the Demo

Execute all benchmark functions with example data:

```bash
poetry run main
```

This will run demonstrations of all available modules and display their outputs.

Using Individual Modules

```python
from llm_benchmark.algorithms.primes import Primes
from llm_benchmark.control.single import SingleForLoop
from llm_benchmark.datastructures.dslist import DsList
from llm_benchmark.strings.strops import StrOps

# Check if a number is prime
is_prime = Primes.is_prime(17)  # Returns: True

# Sum a range of numbers
total = SingleForLoop.sum_range(10)  # Returns: 45

# Reverse a list
reversed_list = DsList.reverse_list([1, 2, 3, 4, 5])  # Returns: [5, 4, 3, 2, 1]

# Check palindrome
is_palindrome = StrOps.palindrome("racecar")  # Returns: True
```

Testing

Run Unit Tests

Execute all unit tests without benchmarking:

```bash
```

```
poetry run pytest --benchmark-skip tests/
```

Run Benchmarks

Execute performance benchmarks for all functions:

```bash
poetry run pytest --benchmark-only tests/
```

This will measure and compare the execution time of different
implementations and provide detailed performance metrics.

Module Documentation

Algorithms
- `Primes.is_prime(n)` - Check if a number is prime
- `Primes.is_prime_ineff(n)` - Inefficient prime check (for benchmarking)
- `Primes.sum_primes(n)` - Sum all primes from 0 to n
- `Primes.prime_factors(n)` - Get prime factorization
- `Sort.sort_list(v)` - Sort a list in-place
- `Sort.dutch_flag_partition(v, pivot)` - Partition list around pivot
- `Sort.max_n(v, n)` - Find the N largest elements

Control Flow
- `SingleForLoop.sum_range(n)` - Sum numbers from 0 to n
- `SingleForLoop.max_list(v)` - Find maximum in list
- `SingleForLoop.sum_modulus(n, m)` - Sum numbers divisible by m
- `DoubleForLoop.sum_square(n)` - Sum of squares using nested loops
- `DoubleForLoop.sum_triangle(n)` - Triangular number sum
- `DoubleForLoop.count_pairs(v)` - Count unique pairs in list
- `DoubleForLoop.count_duplicates(v1, v2)` - Count duplicates between
lists
- `DoubleForLoop.sum_matrix(m)` - Sum all matrix elements

Data Structures
- `DsList.modify_list(v)` - Add 1 to each element
- `DsList.search_list(v, n)` - Find all indices of value
- `DsList.sort_list(v)` - Sort and return copy
- `DsList.reverse_list(v)` - Reverse and return copy
- `DsList.rotate_list(v, n)` - Rotate list by n positions
- `DsList.merge_lists(v1, v2)` - Merge two lists

String Operations
- `StrOps.str_reverse(s)` - Reverse a string
- `StrOps.palindrome(s)` - Check if string is palindrome

SQL Queries
- `SqlQuery.query_album(name)` - Check if album exists
- `SqlQuery.join_albums()` - Join Album, Artist, and Track tables
- `SqlQuery.top_invoices()` - Get top 10 invoices by total

Project Structure

```
llm-benchmarking-py/
├── src/
│   └── llm_benchmark/
│       ├── algorithms/      # Algorithm implementations
│       ├── control/         # Control flow operations
│       ├── datastructures/  # Data structure operations
│       ├── generator/       # Test data generators
│       ├── sql/            # SQL query operations
│       └── strings/        # String manipulation
├── tests/                   # Unit tests and benchmarks
├── data/                    # Database files (Chinook SQLite)
├── main.py                 # Demo script
└── pyproject.toml          # Project configuration
```

Development

Code Formatting
```bash
poetry run black src/ tests/
poetry run isort src/ tests/
```

License

See project repository for license information.

Contributing

Contributions are welcome! Please ensure all tests pass before submitting pull requests.

Author

Matthew Truscott (matthew.truscott@turintech.ai)